SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
ProgramName: B. Tech		**Assignment Type: Lab**	**AcademicYear:** 2025-2026
CourseCoordinatorName		Venkataramana Veeramsetty	
Instructor(s)Name		Dr. V. Venkataramana (Co-ordinator) Dr. T. Sampath Kumar Dr. Pramoda Patro Dr. Brij Kishor Tiwari Dr.J.Ravichander Dr. Mohammand Ali Shaik Dr. Anirodh Kumar Mr. S.Naresh Kumar Dr. RAJESH VELPULA Mr. Kundhan Kumar Ms. Ch.Rajitha Mr. M Prakash Mr. B.Raju Intern 1 (Dharma teja) Intern 2 (Sai Prasad) Intern 3 (Sowmya) NS_2 (Mounika)	
CourseCode	24CS002PC215	**CourseTitle**	AI Assisted Coding
Year/Sem	II/I	**Regulation**	R24
Date and Day of Assignment	Week4 - Tuesday	**Time(s)**	
Duration	2 Hours	**Applicableto Batches**	
AssignmentNumber: **8.2**(Present assignment number)/**24**(Total number of assignments)			
NAME: VADDEM CHANDANA			
HALL TICKET NO: 2403A51186			

Q.No.	Question	*ExpectedTime to complete*
1	Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases **Lab Objectives:** • To introduce students to test-driven development (TDD) using AI code generation tools. • To enable the generation of test cases before writing code implementations.	Week4 - Wednesday

- To reinforce the importance of testing, validation, and error handling.
- To encourage writing clean and reliable code based on AI-generated test expectations.

Lab Outcomes (LOs):
After completing this lab, students will be able to:

- Use AI tools to write test cases for Python functions and classes.
- Implement functions based on test cases in a test-first development style.
- Use unittest or pytest to validate code correctness.
- Analyze the completeness and coverage of AI-generated tests.
- Compare AI-generated and manually written test cases for quality and logic

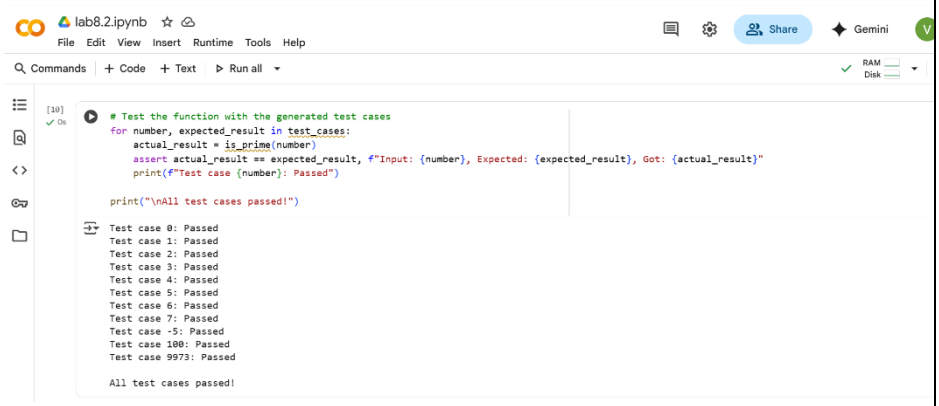
Task Description#1

Use AI to generate test cases for a function is_prime(n) and then implement the function.

Requirements:

- Only integers > 1 can be prime.
- Check edge cases: 0, 1, 2, negative numbers, and large primes.

CODE AND OUT PUT:



Expected Output#1

- A working prime checker that passes AI-generated tests using edge coverage.

Task Description#2 (Loops)

- Ask AI to generate test cases for celsius_to_fahrenheit(c) and fahrenheit_to_celsius(f).

Requirements

- Validate known pairs: 0°C = 32°F, 100°C = 212°F.
- Include decimals and invalid inputs like strings or None

CODE AND OUTPUT:

```python
# Test cases for celsius_to_fahrenheit(c)
celsius_test_cases = [
    (0, 32.0),     # Known pair
    (100, 212.0), # Known pair
    (25, 77.0),
    (-10, 14.0),
    (0.5, 32.9),  # Decimal input
    (-20.3, -4.54), # Decimal input
    ('abc', None), # Invalid input (string)
    (None, None),  # Invalid input (None)
    ([], None),    # Invalid input (list)
    ({}, None)     # Invalid input (dictionary)
]

# Test cases for fahrenheit_to_celsius(f)
fahrenheit_test_cases = [
    (32, 0.0),     # Known pair
    (212, 100.0), # Known pair
    (77, 25.0),
    (14, -10.0),
    (32.9, 0.5),  # Decimal input
    (-4.54, -20.3), # Decimal input
    ('xyz', None), # Invalid input (string)
    (None, None),  # Invalid input (None)
    ((), None),    # Invalid input (tuple)
    (set(), None)  # Invalid input (set)
    (set(), None)  # Invalid input (set)
]

print("Celsius to Fahrenheit Test Cases:")
for input_c, expected_f in celsius_test_cases:
    print(f"Input: {input_c}, Expected: {expected_f}")

print("\nFahrenheit to Celsius Test Cases:")
for input_f, expected_c in fahrenheit_test_cases:
    print(f"Input: {input_f}, Expected: {expected_c}")
```

```
Celsius to Fahrenheit Test Cases:
Input: 0, Expected: 32.0
Input: 100, Expected: 212.0
Input: 25, Expected: 77.0
Input: -10, Expected: 14.0
Input: 0.5, Expected: 32.9
Input: -20.3, Expected: -4.54
Input: abc, Expected: None
Input: None, Expected: None
Input: [], Expected: None
Input: {}, Expected: None

Fahrenheit to Celsius Test Cases:
Input: 32, Expected: 0.0
Input: 212, Expected: 100.0
Input: 77, Expected: 25.0
Input: 14, Expected: -10.0
```

```
Fahrenheit to Celsius Test Cases:
Input: 32, Expected: 0.0
Input: 212, Expected: 100.0
Input: 77, Expected: 25.0
Input: 14, Expected: -10.0
Input: 32.9, Expected: 0.5
Input: -4.54, Expected: -20.3
Input: xyz, Expected: None
Input: None, Expected: None
Input: (), Expected: None
Input: set(), Expected: None
```

Expected Output#2

Dual conversion functions with complete test coverage and safe type handling

Task Description#3

Use AI to write test cases for a function count_words(text) that returns the number of words in a sentence.

Requirement

Handle normal text, multiple spaces, punctuation, and empty strings.

CODE AND OUTPUT:

```
# Test cases for count_words(text)
count_words_test_cases = [
    ("Hello world", 2),                    # Normal text
    ("This is a sentence with five words.", 6), # Normal text with punctuation
    ("  Leading and trailing spaces  ", 4), # Leading and trailing spaces
    ("Multiple   spaces   between   words", 4), # Multiple spaces between words
    ("", 0),                               # Empty string
    ("    ", 0),                           # String with only spaces
    ("Word with, punctuation!", 3),        # Punctuation attached to words
    (" One . Two , Three ; Four : Five ! ", 5), # Punctuation with spaces
    ("A sentence with\nnewline characters", 4), # Newline characters
    ("A sentence with\ttab characters", 4),   # Tab characters
    ("  Hello   world!   ", 2)             # Mixed spaces and punctuation
]

print("count_words Test Cases:")
for text, expected_count in count_words_test_cases:
    print(f"Input: '{text}', Expected: {expected_count}")
```

```
count_words Test Cases:
Input: 'Hello world', Expected: 2
Input: 'This is a sentence with five words.', Expected: 6
Input: '  Leading and trailing spaces  ', Expected: 4
Input: 'Multiple   spaces   between   words', Expected: 4
Input: '', Expected: 0
Input: '    ', Expected: 0
Input: 'Word with, punctuation!', Expected: 3
Input: 'Word with, punctuation!', Expected: 3
Input: ' One . Two , Three ; Four : Five ! ', Expected: 5
Input: 'A sentence with
newline characters', Expected: 4
Input: 'A sentence with tab characters', Expected: 4
Input: '  Hello   world!   ', Expected: 2
```

Expected Output#3

Accurate word count with robust test case validation.

Task Description#4

- Generate test cases for a BankAccount class with:

  Methods:

  deposit(amount)

  withdraw(amount)

  check_balance()

Requirements:

- Negative deposits/withdrawals should raise an error.
- Cannot withdraw more than balance.

CODE AND OUTPUT:

```
# Generate test cases for BankAccount class
bank_account_test_cases = [
    # Test deposit
    {"method": "deposit", "amount": 100, "expected_balance": 100, "description": "Initial deposit"},
    {"method": "deposit", "amount": 50.5, "expected_balance": 150.5, "description": "Deposit with decimal"},
    {"method": "deposit", "amount": -10, "expected_error": ValueError, "description": "Negative deposit"},
    {"method": "deposit", "amount": 0, "expected_balance": 150.5, "description": "Zero deposit"},

    # Test withdraw
    {"method": "withdraw", "amount": 50, "expected_balance": 100.5, "description": "Valid withdrawal"},
    {"method": "withdraw", "amount": 100.5, "expected_balance": 0, "description": "Withdraw full balance"},
    {"method": "withdraw", "amount": 1, "expected_error": ValueError, "description": "Withdraw more than balance"},
    {"method": "withdraw", "amount": -20, "expected_error": ValueError, "description": "Negative withdrawal"},
    {"method": "withdraw", "amount": 0, "expected_balance": 0, "description": "Zero withdrawal"},

    # Test check_balance (implicitly tested by deposit and withdraw)
    # Add explicit check_balance tests if needed
]

print("Test cases for BankAccount:")
# Note: The expected_balance for withdrawal tests is based on the balance *after* the deposit tests have run.
# A more robust approach would be to reset the bank account state for each test or use a testing framework.
for test_case in bank_account_test_cases:
    print(f"Method: {test_case['method']}, Amount: {test_case.get('amount', 'N/A')}, Expected: {test_case.get('expected_balance', test_case.get('expected_error'))}, Description:
```

```
Test cases for BankAccount:
Method: deposit, Amount: 100, Expected: 100, Description: Initial deposit
Method: deposit, Amount: 50.5, Expected: 150.5, Description: Deposit with decimal
Method: deposit, Amount: -10, Expected: <class 'ValueError'>, Description: Negative deposit
Method: deposit, Amount: 0, Expected: 150.5, Description: Zero deposit
```

```
Test cases for BankAccount:
Method: deposit, Amount: 100, Expected: 100, Description: Initial deposit
Method: deposit, Amount: 50.5, Expected: 150.5, Description: Deposit with decimal
Method: deposit, Amount: -10, Expected: <class 'ValueError'>, Description: Negative deposit
Method: deposit, Amount: 0, Expected: 150.5, Description: Zero deposit
Method: withdraw, Amount: 50, Expected: 100.5, Description: Valid withdrawal
Method: withdraw, Amount: 100.5, Expected: 0, Description: Withdraw full balance
Method: withdraw, Amount: 1, Expected: <class 'ValueError'>, Description: Withdraw more than balance
Method: withdraw, Amount: -20, Expected: <class 'ValueError'>, Description: Negative withdrawal
Method: withdraw, Amount: 0, Expected: 0, Description: Zero withdrawal
```

Expected Output#4

- AI-generated test suite with a robust class that handles all test cases.

Task Description#5

Generate test cases for is_number_palindrome(num), which checks if an integer reads the same backward.

Examples:

121 → True
123 → False
0, negative numbers → handled gracefully

CODE AND OUTPUT:

```
# Test cases for is_number_palindrome(num)
is_number_palindrome_test_cases = [
    (121, True),      # Positive palindrome
    (123, False),     # Positive non-palindrome
    (0, True),        # Edge case: 0 (considered a palindrome)
    (1, True),        # Single digit (considered a palindrome)
    (11, True),       # Two-digit palindrome
    (10, False),      # Two-digit non-palindrome
    (1221, True),     # Even number of digits palindrome
    (12321, True),    # Odd number of digits palindrome
    (12345, False),   # Longer non-palindrome
    (-121, False),    # Negative number (often considered not a palindrome in this context)
    (-1, False),      # Negative single digit
    (1001, True),     # Palindrome with zeros
    (10, False)
]

print("is_number_palindrome Test Cases:")
for num, expected_result in is_number_palindrome_test_cases:
    print(f"Input: {num}, Expected: {expected_result}")
```

```
is_number_palindrome Test Cases:
Input: 121, Expected: True
Input: 123, Expected: False
Input: 0, Expected: True
Input: 1, Expected: True
Input: 11, Expected: True
Input: 10, Expected: False
Input: 1221, Expected: True
Input: 12321, Expected: True
Input: 12345, Expected: False
Input: -121, Expected: False
Input: -1, Expected: False
Input: 1001, Expected: True
Input: 10, Expected: False
```

Expected Output#5

- Number-based palindrome checker function validated against test cases.

Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots

Evaluation Criteria:

Criteria	Max Marks
Task #1	0.5
Task #2	0.5
Task #3	0.5
Task #4	0.5
Task #5	0.5
Total	**2.5 Marks**